

Manual de Usuario

API de Empleados - Spring Boot

Mateo Benjamín Ortiz de Leon

2024204

IN5CM

Cosas necesarias para comenzar

Nos aseguraremos de tener los siguientes programas y archivos:

- JDK 21
- MySQL
- IntelliJ IDEA
- Postman

Configurar la base de datos

1. Crear la base de datos en MySQL

ejecutar los siguientes comandos en MySQL Workbench:

```
DROP DATABASE IF EXISTS DBRepuestosAutomotriz_in5cm;  
CREATE DATABASE DBRepuestosAutomotriz_in5cm;  
USE DBRepuestosAutomotriz_in5cm;
```

2. Crear la tabla de empleados

```
CREATE TABLE Empleados(  
    id_empleado INT AUTO_INCREMENT NOT NULL,  
    nombre_empleado VARCHAR(60) NOT NULL,  
    apellido_empleado VARCHAR(60) NOT NULL,  
    puesto_empleado VARCHAR(20) NULL,  
    email_empleado VARCHAR(100) NOT NULL,  
    PRIMARY KEY (id_empleado)  
);
```

3. Insertar registros de prueba

```
INSERT INTO Empleados VALUES  
(null, 'Carlos', 'Pérez', 'Vendedor', 'carlos@empresa.com'),  
(null, 'Ana', 'López', 'Cajera', 'ana@empresa.com'),  
(null, 'Luis', 'Martínez', 'Administrador', 'luis@empresa.com');
```

4. Configurar las propiedades

En tu proyecto edita las propperties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/DBRepuestosAutomotriz_in5cm  
spring.datasource.username=root  
spring.datasource.password=tu_contraseña  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true
```

Funcionalidad del código

Empleado.java (El modelo)

Esta clase representa la entidad empleado. define cómo se ve un empleado en Java y cómo se mapea a la tabla de MySQL.

```
@Entity
@Table(name = "Empleados")
public class Empleado {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_empleado")
    private Integer idEmpleado;

    @Column(name = "nombre_empleado")
    private String nombreEmpleado;

    // + getters y setters
}
```

Las anotaciones (@Entity, @Table, etc.) le dicen a Spring cómo conectar esto con la base de datos.

EmpleadoRepository.java

Esta es una interfaz que extiende JpaRepository. Básicamente ya trae métodos para guardar, buscar, actualizar y eliminar sin que tengas que escribir SQL.

```
@Repository
public interface EmpleadoRepository extends JpaRepository<Empleado, Integer> {
}
```

Eso es todo. Spring hace el resto automáticamente.

EmpleadoService y EmpleadoServiceImpls

Aquí va la lógica del negocio. Los métodos importantes son:

- getAllEmpleados() - trae todos los empleados
- getEmpleadoById(id) - busca uno por ID
- saveEmpleado(empleado) - guarda uno nuevo
- updateEmpleado(id, empleado) - actualiza uno existente
- deleteEmpleado(id) - elimina uno

Ejemplo del método update:

```
public Empleado updateEmpleado(Integer id, Empleado empleado) {
    Empleado empleadoExistente = empleadoRepository.findById(id).orElse(null);
    if(empleadoExistente == null) return null;

    empleadoExistente.setNombreEmpleado(empleado.getNombreEmpleado());
}
```

```
    empleadoExistente.setApellidoEmpleado(empleado.getApellidoEmpleado());
    empleadoExistente.setPuestoEmpleado(empleado.getPuestoEmpleado());
    empleadoExistente.setEmailEmpleado(empleado.getEmailEmpleado());

    return empleadoRepository.save(empleadoExistente);
}
```

Primero busca el empleado, luego actualiza sus datos y lo guarda.

EmpleadoController.java

Este es el controlador que maneja las peticiones HTTP. Define los endpoints de la API.

Los endpoints que tengo son:

- GET /api/empleados - ver todos
- GET /api/empleados/{id} - ver uno
- POST /api/empleados - crear uno
- PUT /api/empleados/{id} - actualizar uno
- DELETE /api/empleados/{id} - eliminar uno

Ejemplo del GET por ID:

```
@GetMapping("/{id}")
public ResponseEntity<Empleado> getEmpleadoById(@PathVariable Integer id){
    Empleado empleado = empleadoService.getEmpleadoById(id);
    return ResponseEntity.ok(empleado);
}
```

El @PathVariable agarra el ID de la URL y lo pasa al método.

Cómo probar en Postman

Aquí viene lo bueno. Voy a explicar cómo probar cada endpoint.

Paso 0: Iniciar la aplicación

1. Abre el proyecto en IntelliJ
2. Corre la aplicación (botón play verde)
3. Espera a que diga "Started RepuestosYAutomotricesApplication"
4. Abre Postman

Prueba 1: Ver todos los empleados (GET)

En Postman:

- Método: GET
- URL: `http://localhost:8080/api/empleados`
- Dale a "Send"

Debería devolverte algo como:

```
[
  {
    "idEmpleado": 1,
    "nombreEmpleado": "Carlos",
    "apellidoEmpleado": "Pérez",
    "puestoEmpleado": "Vendedor",
    "emailEmpleado": "carlos@empresa.com"
  },
  ...
]
```

Prueba 2: Buscar un empleado por ID (GET)

- Método: GET
- URL: `http://localhost:8080/api/empleados/1`
- (cambia el 1 por el ID que quieras)

Te devuelve ese empleado específico.

Prueba 3: Crear un empleado nuevo (POST)

Esta es la que más lío da, pero es fácil:

5.

1. En Postman, pon:

- Método: POST
- URL: `http://localhost:8080/api/empleados`

6.

2. Ve a la pestaña "Headers" y agrega:

- Key: Content-Type
- Value: application/json

7.

3. Ve a la pestaña "Body":

- Selecciona "raw"
- Selecciona "JSON" en el dropdown
- Pega esto:

```
{
  "nombreEmpleado": "Mateo",
  "apellidoEmpleado": "Ortiz",
  "puestoEmpleado": "Gerente",
  "emailEmpleado": "mortiz-202420204@kinal.edu.gt"
}
```

8.

4. "Send"

Te debería devolver el empleado creado con su ID nuevo:

```
{
  "idEmpleado": 4,
  "nombreEmpleado": "Mateo",
  ...
}
```

Prueba 4: Actualizar un empleado (PUT)

Digamos que quieres cambiar el puesto del empleado con ID 4:

- - Método: PUT
- URL: http://localhost:8080/api/empleados/4
- Headers: Content-Type = application/json
- Body (raw - JSON):

```
{
  "nombreEmpleado": "Mateo",
  "apellidoEmpleado": "Ortiz",
  "puestoEmpleado": "Director",
  "emailEmpleado": "mortiz-202420204@kinal.edu.gt"
}
```

Cambié "Gerente" por "Director". Te devuelve el empleado actualizado.

Prueba 5: Eliminar un empleado (DELETE)

- Método: DELETE
- URL: <http://localhost:8080/api/empleados/4>
- Dale "Send"

Si todo sale bien, te da un 200 OK y el empleado ya no existe.

Para verificar que se borró, haz un GET a [/api/empleados/4](#) y debería decir null o dar error.

Errores que me han salido (y cómo los arreglé)

Error: Connection refused

Problema: La app no está corriendo

Solución: Corre la aplicación en IntelliJ y espera a que inicie bien

Error: 404 Not Found

Problema: Escribiste mal la URL

Solución: Verifica que sea `http://localhost:8080/api/empleados`

Error: 400 Bad Request (en POST o PUT)

Problema: El JSON está mal escrito o falta el header

Solución:

- • Revisa que el JSON esté bien (comillas dobles, comas bien puestas)
- • Asegúrate de tener Content-Type: application/json en Headers

Error: Failed to configure a DataSource

Problema: No se puede conectar a MySQL

Solución:

- • Verifica que MySQL esté corriendo
- • Checa que el application.properties tenga la contraseña correcta
- • Asegúrate de que la base de datos existe

Consejos finales

- Si algo falla, mira la consola de IntelliJ. Ahí dice qué pasó.
- Guarda tus peticiones en Postman (haz una Collection) para no tener que escribirlas de nuevo.
- Prueba primero el GET para ver que todo funciona antes de hacer POST o PUT.
- No incluyas el ID cuando crees un empleado. MySQL lo genera automático.
- Si cambias algo en el código, reinicia la aplicación.